# Atlas Platform - Comprehensive System Architecture Document

**Version:** 2.0
**Date:** April 10, 2026
**Status:** Approved
**Prepared by:** System Architecture Team (Amit, Aditya, Prishav)
**Target Audience:** Engineering Team, Law Enforcement System Integrators, Auditors

---

## 1. Executive Summary

The Atlas Platform is a highly specialized, secure, and modern microservices-oriented software system developed to automate the ingestion, classification, review, and analysis of First Information Reports (FIRs) and other sensitive law enforcement documents for the Gujarat Police.

Faced with a strict 11-month timeline and a lean 2-developer team, the architecture prioritizes rapid development, deployment simplicity, and robust, completely offline Natural Language Processing (NLP) capabilities. Due to the confidential classification of police FIR data and legal mandates like BNS §73 / Section 228A CrPC (which prohibit disclosing victim identities in sexual offence cases), the platform must operate in a completely air-gapped environment. It cannot rely on cloud-based APIs like OpenAI or Google Cloud NLP. Instead, it relies on locally hosted, fine-tuned models operating within strict resource constraints (e.g., maximum 12 GB VRAM per district hub server).

This extensive document acts as the definitive source of truth for the platform's technical architecture. It covers the high-level design, presentation and application tiers, polyglot data storage schemas, detailed Natural Language Processing workflows, the complex security and Role-Based Access Control (RBAC) matrix, and the Machine Learning Operations (MLOps) lifecycle.

---

## 2. High-Level Architectural Design and Philosophy

### 2.1 The Modular Monolith Approach

As detailed in Architectural Decision Record **ADR-D01**, the team actively chose a **Modular Monolith** architecture over a distributed microservices array. While the codebase is logically separated into distinct domains (e.g., Auth, FIRs, Ingestion, NLP, Dashboard), the backend executes as a single FastAPI Python application.

**Strategic Rationale:**

- **Deployment Simplicity:** The platform is deployed to 6 separate district hub servers via a unified `docker-compose.yml` file. Local police stations access the platform via their browsers with zero local installation overhead.
- **Resource Constraints:** Splitting the NLP model loading across different microservices would lead to duplicate RAM/VRAM usage. A monolith allows the heavy IndicBERT/MuRIL and fastText models to be loaded into memory once and shared across all request threads.
- **Team Velocity:** For a 2-person team, tracing bugs across distributed services, managing message brokers, and dealing with eventual consistency would severely impact the 11-month delivery timeline. A modular monolith provides the structure of microservices without the operational tax.

### 2.2 Core System Components

The Atlas Platform is constructed from five primary tiers:

1. **Presentation Tier:** A Next.js (React) Single Page Application (SPA) providing dashboards, review interfaces, and search capabilities.
2. **Application Tier:** A FastAPI (Python) backend handling routing, authentication, business logic, and orchestration of ML tasks.
3. **Data Tier:** A polyglot persistence layer utilizing PostgreSQL (relational structured data), MongoDB (raw unstructured data lake), and Redis (in-memory caching).
4. **Machine Learning Tier:** Embedded local NLP pipelines handling language detection (fastText), text transliteration (IndicXlit), and zero-shot/fine-tuned classification.
5. **MLOps Tier:** A parallel suite of tools including Label Studio (for human-in-the-loop data annotation) and MLflow (for tracking experiment parameters and model weights).

### 2.3 System Interaction Diagram

Below is the high-level interaction diagram demonstrating how clients, the backend, databases, and monitoring tools communicate within the Docker Compose network.

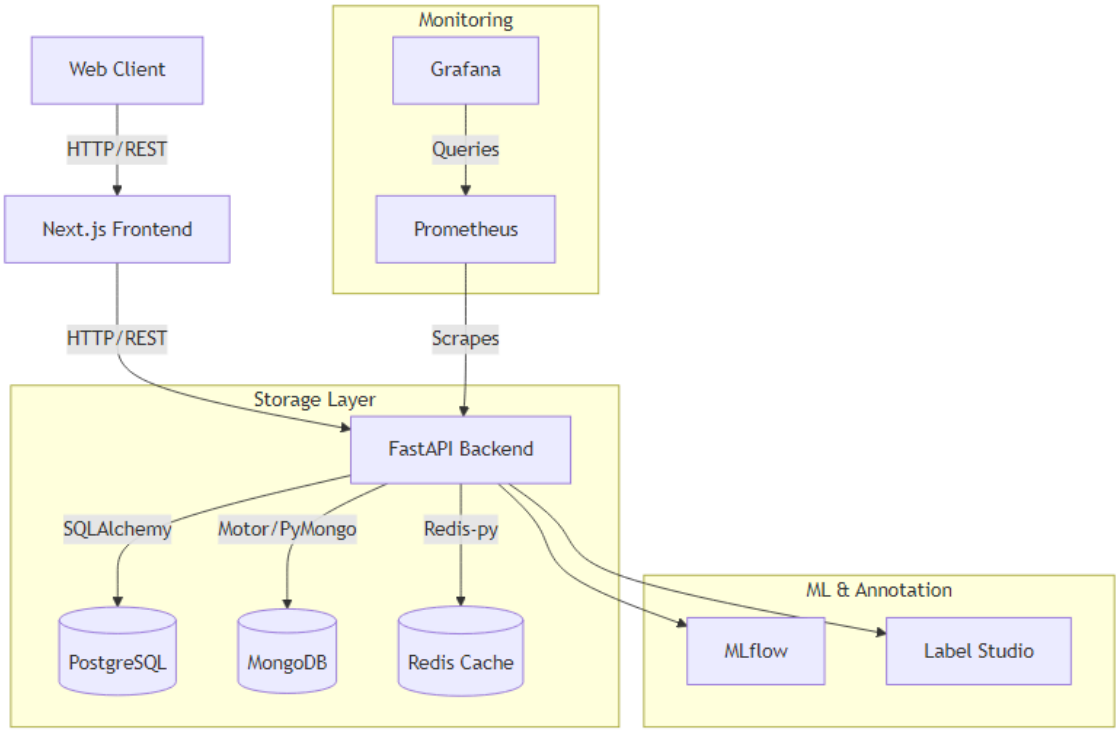



*Figure 1: High-Level Component Interaction Diagram showing the Web Client routing through the Next.js Frontend to the FastAPI Backend, which interfaces with polyglot storage, ML infrastructure, and observability tools.*

---

## 3. Presentation Tier (Frontend Architecture)

The frontend is engineered to be highly responsive, accessible, and easily maintainable. It serves as the primary interface for Investigating Officers (IOs), Station House Officers (SHOs), and higher-ranking officials.

### 3.1 Technology Stack

- **Core Framework:** Next.js 14.2 (React 18) utilizing the modern `app/` router paradigm.
- **Language:** TypeScript 5+ for strict type safety and interface definitions.

- **Styling:** Tailwind CSS combined with `shadcn/ui`. This provides a highly customizable, utility-first styling approach without the overhead of heavy component libraries.
- **State Management:** React Context API for global state (e.g., user session, RBAC roles), coupled with local component state.
- **Data Fetching:** Centralized API client ( `src/lib/api.ts` ) managing REST HTTP requests to the FastAPI backend, implementing automatic JWT token injection and refresh logic.

### 3.2 Key User Interfaces

1. **Authentication Portal:** Secure login screen handling JWT negotiation.
2. **IO Dashboard:** A district-scoped view showing recent FIRs, pending OCR jobs, and AI-flagged classification mismatches.
3. **FIR Review Interface:** A detailed side-by-side view showing the original uploaded PDF alongside the AI-extracted structured data. Officers can accept the AI's classification or manually override it.
4. **Global Analytics (SP/ADMIN):** Multi-district heatmaps and statistical breakdowns of crime categories across the state.

---

## 4. Application Tier (Backend Architecture)

The Application Tier acts as the central orchestrator. Built on Python 3.9+, it leverages FastAPI to provide extremely high throughput and automatic Swagger/OpenAPI documentation.

### 4.1 Framework and Concurrency Model

- **FastAPI & Uvicorn:** FastAPI utilizes Python's ASGI (Asynchronous Server Gateway Interface) standard. This allows the API to handle thousands of concurrent connections.
- **Thread Pool Executors:** Because ML inference (like running a 110M parameter IndicBERT model) is CPU/GPU bound and blocking, these operations are explicitly offloaded to `asyncio.run_in_executor`. This prevents a single heavy classification request from stalling the entire web server, ensuring the UI remains snappy for other users.

### 4.2 API Router Organization

The codebase ( `backend/app/api/v1/` ) is highly modularized:

- `auth.py` : Handles login, password hashing (bcrypt via pgcrypto/passlib), and JWT issuance.
- `firs.py` : Provides paginated `GET` endpoints, strict `PATCH` endpoints for updating status, and role-filtered queries.
- `ingest.py` : Exposes `POST /api/v1/ingest` , the massive entrypoint for uploading raw FIR PDFs.
- `predict.py` : Provides stateless endpoints ( `POST /predict/classify` ) allowing clients to test text against the NLP models without committing to the database.
- `dashboard.py` : Runs complex SQL aggregation queries to power frontend charts.

### 4.3 Middlewares and Telemetry

- **CORS Middleware:** Strictly configured to accept requests only from trusted frontend origins (e.g., `http://localhost:3000` ).
- **Prometheus Instrumentator:** Wraps the FastAPI app to automatically expose a `/metrics` endpoint. This tracks HTTP request durations, status codes, and endpoint popularity, which are subsequently scraped by Prometheus.
- **Structured Logging:** Utilizes `structlog` to output JSON-formatted logs. Every log entry includes an ISO timestamp, log level, module name, and contextual data (like `fir_id` or `user_id` ), making it trivial to search through logs in a centralized logging stack.

---

# 5. Data Tier & Polyglot Schema Design

The Atlas Platform recognizes that law enforcement data comes in two distinct forms: highly structured metadata (dates, names, legal sections) and messy, unpredictable unstructured data (raw narrative text, OCR noise, original PDFs). To handle this gracefully, the platform uses Polyglot Persistence.

## 5.1 Relational Storage (PostgreSQL 15)

PostgreSQL is the absolute source of truth for the system's structured operational data. It is accessed via SQLAlchemy ORM, and schema versioning is managed by Alembic.

### 5.1.1 Entity Relationship Diagram

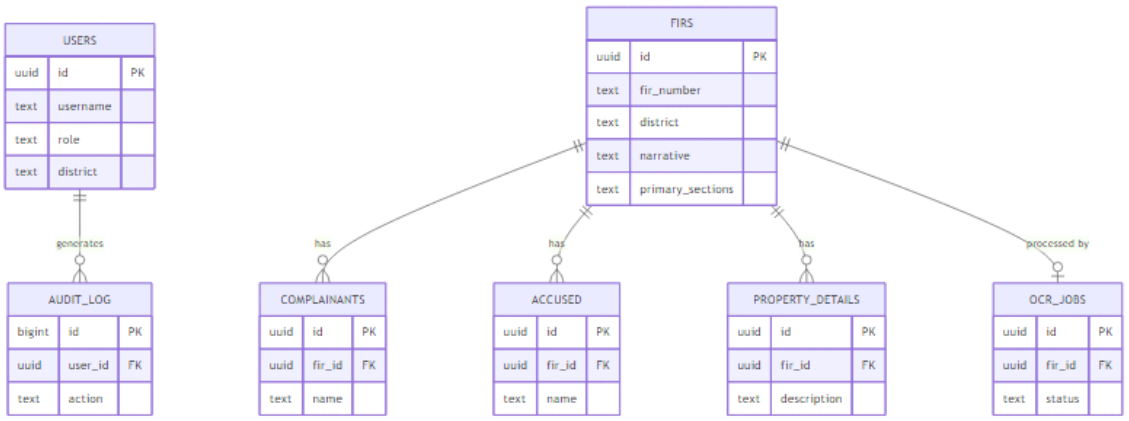



*Figure 2: Entity Relationship Diagram highlighting the central `FIRS` table and its one-to-many relationships with `COMPLAINANTS`, `ACCUSED`, and `PROPERTY_DETAILS`.*

### 5.1.2 Core Tables Breakdown ( `001_initial_schema.py` )

- **`firs`** : The massive central hub. It stores critical metadata such as `fir_number`, `police_station`, `district`, arrays of `primary_sections`, and the `completeness_pct`. Crucially, it stores the AI-extracted `narrative` text.
- **Child Tables ( `complainants`, `accused`, `property_details` )**: These tables hold a foreign key ( `fir_id` ) linking back to the `firs` table. They enforce cascading deletes to ensure no orphaned records remain if a FIR is purged.
- **`users`** : Stores authentication and authorization data, including the user's `district` and specific `role`.
- **`audit_log`** : An immutable ledger tracking every significant write or patch operation. It records the `user_id`, `action`, `resource_id`, and the client's `ip_address`, fulfilling strict police auditing requirements.
- **`ocr_jobs`** : Manages the state of asynchronous background tasks parsing complex, multi-page scanned documents.

## 5.2 Unstructured Data Lake (MongoDB 7)

MongoDB operates alongside PostgreSQL as an unstructured data lake ( `atlas_raw` database).

- **The Philosophy of Non-Destruction:** When a user uploads a PDF, the backend's `ingest.py` immediately pushes the raw binary and the initial raw text dump into MongoDB.
- **Future-Proofing:** If the NLP models are drastically improved in Year 2, the team can query MongoDB to retrieve the original, untampered raw text of thousands of FIRs and re-run the ingestion pipeline to extract

better data, without worrying about data loss during the initial flawed parsing phase.

### 5.3 In-Memory Cache (Redis 7)

Redis is utilized to accelerate the platform.

- **Dashboard Caching:** Complex SQL aggregations (e.g., "Count of all cybercrimes per district over the last 6 months") are expensive. The results are cached in Redis with a 5-minute Time-To-Live (TTL).
- **Session Management / Rate Limiting:** Redis can quickly track JWT token blacklists and enforce API rate limits to prevent denial-of-service attacks.

---

## 6. Security Architecture and Role-Based Access Control (RBAC)

Security is not an afterthought in the Atlas Platform; it is deeply embedded into the application's core logic via ADR-D03. The system handles highly confidential data, including the identities of victims of sexual violence.

### 6.1 The RBAC Matrix

The system implements a rigid hierarchical structure enforced by the `require_role()` dependency injection in FastAPI.

| Role | Scope | Write Permissions | Read Permissions |
|---|---|---|---|
| **IO** (Investigating Officer) | District-scoped | Upload FIRs, Edit Own | Own District |
| **SHO** (Station House Officer) | District-scoped | Approve FIRs, Override AI | Own District |
| **DYSP** (Deputy Superintendent) | Multi-station | Approve Charge Sheets | Cross-District |
| **SP** (Superintendent of Police) | District-wide | User Management | Entire District |
| **ADMIN** | System-wide | Global Settings, DB Mgmt | Global |
| **READONLY** | System-wide | None | Global (Heavily Masked) |

### 6.2 District Scoping at the Database Layer

To prevent a compromised IO account from scraping the entire state's database, FastAPI automatically inspects the JWT token. If the user is an `IO` or `SHO`, the backend silently appends `.filter(firs.c.district == user_district)` to all SQLAlchemy queries. This is executed at the database level, ensuring data leakage is physically impossible.

### 6.3 Programmatic PII Masking and Victim Protection

By law (BNS §73 / Section 228A CrPC), the identity of a victim of sexual offences cannot be disclosed.

- **Trigger Condition:** The system intercepts any FIR where `primary_sections` contains codes mapping to sexual offences (e.g., BNS §63-99, IPC §376).
- **Unconditional Masking:** `core/pii.py` activates and forcefully overwrites the `complainant_name` and `place_address` fields with `[VICTIM-PROTECTED]` and `[ADDRESS-PROTECTED]`.
- **Rank Agnostic:** This masking is applied *before* the JSON leaves the server and applies to **all roles**, including the Global ADMIN. The true data remains safely encrypted in the PostgreSQL database, but is never transmitted over the network for these specific cases.

---

## 7. The FIR Ingestion & Parsing Pipeline

The document ingestion pipeline ( `app/ingestion/pipeline.py` ) is the critical bridge translating chaotic, real-world police documents into clean, structured, analyzable data.

### 7.1 Sequence of Ingestion Operations

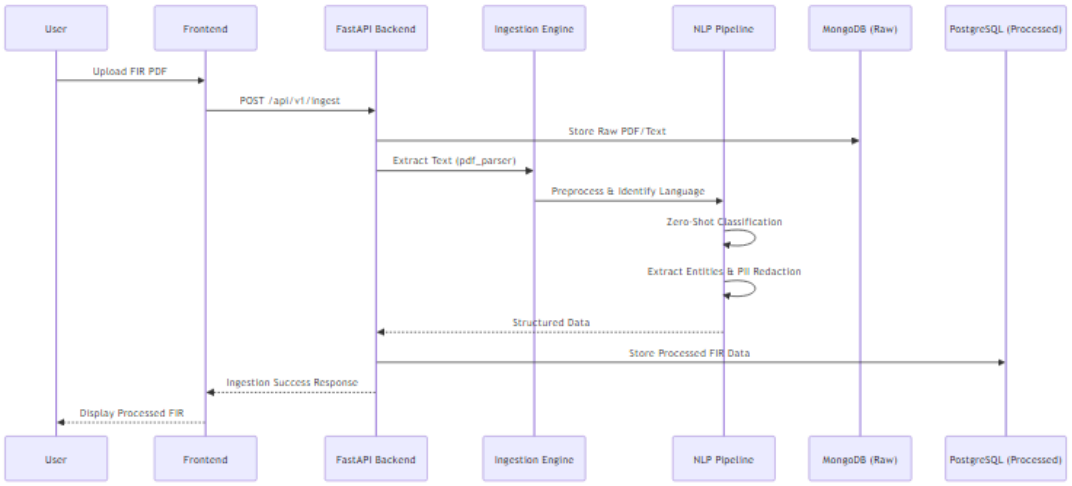



*Figure 3: Sequence Diagram illustrating the synchronous and asynchronous flow of data from the User through the API, Database, Ingestion Engine, and NLP Pipeline.*

### 7.2 Detailed Steps

1. **File Upload:** The Next.js client posts a multipart form containing the FIR PDF to `POST /api/v1/ingest` .
2. **Raw Extraction ( `pdf_parser.py` ):** The system extracts UTF-8 string data from the PDF layout.
3. **Regex & Heuristic Parsing ( `fir_parser.py` ):** Complex regular expressions scan the text block to isolate discrete fields. For example, it searches for `"FIR No:"` followed by digits, or `"Date of Occurrence:"` followed by timestamps.
4. **Data Flattening:** The deeply nested dictionaries returned by the parser are flattened into a single-level dictionary that exactly matches the PostgreSQL `firs` table schema.
5. **Fallback Guarantees:** The pipeline is fault-tolerant. If the parser completely fails to identify the distinct `narrative` section of the FIR, it falls back to using the entire `raw_text` as the narrative. This ensures the AI classification models always receive text to analyze, preventing silent ingestion failures.

---

## 8. Natural Language Processing (NLP) & AI Architecture

The primary mandate of the Atlas Platform is to accurately classify multilingual FIR text into 11 standardized crime categories (e.g., `murder` , `theft` , `cybercrime` , `narcotics` ).

As outlined in ADR-D02 and ADR-D06, the reality of police data is messy: a single FIR narrative might start with formal English, switch to native Gujarati script, feature Romanised Gujarati spelling (e.g., typing "amdavad" instead of "અમદાવાદ"), and quote legal sections in Hindi.

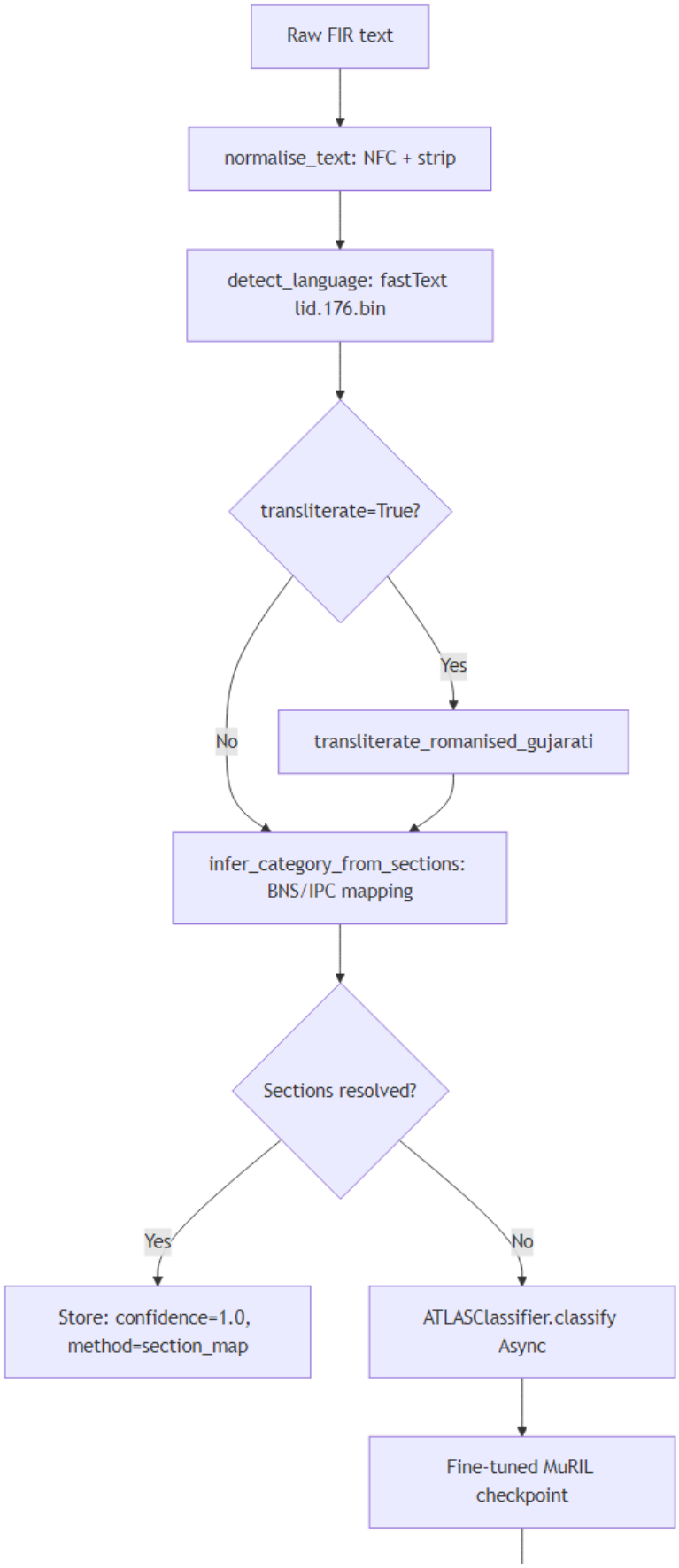
### 8.1 The Multilingual Preprocessing Pipeline

Before any classification occurs, the text is heavily cleaned:

1. **Unicode Normalization:** Strips out strange invisible characters introduced by Tesseract OCR.
2. **Language Detection ( `language.py` ):** Utilizes Facebook's `fastText` ( `lid.176.bin` , a hyper-fast 126MB CPU model) to identify the primary language of the text block in under 100 milliseconds.
3. **Transliteration ( `IndicXlit` ):** If the text is detected as Roman characters but is phonetically Gujarati, the IndicXlit library maps the English characters back to native Gujarati script. This step is critical because the downstream AI models were trained primarily on native script vocabularies.
4. **Tokenization:** IndicNLP splits the massive block of text into discrete sentences, which are then passed through HuggingFace's `AutoTokenizer` and truncated to a maximum of 512 tokens to fit into the AI's context window.

### 8.2 The Four-Tier Classification Cascade

In Sprint 3, the team discovered that fine-tuning a small language model on just 200 synthetic examples led to "mode collapse"—the AI started guessing the same category for every FIR. To build a robust system, the architecture was drastically altered in Sprint 4 to use a four-tier fallback cascade ( `app/nlp/classify.py` ).


Raw FIR text
normalise_text: NFC + strip
detect_language: fastText lid.176.bin
transliterate=True?
Yes
No
transliterate_romanised_gujarati
infer_category_from_sections: BNS/IPC mapping
Sections resolved?
Yes
No
Store: confidence=1.0, method=section_map
ATLASClassifier.classify Async
Fine-tuned MuRIL checkpoint

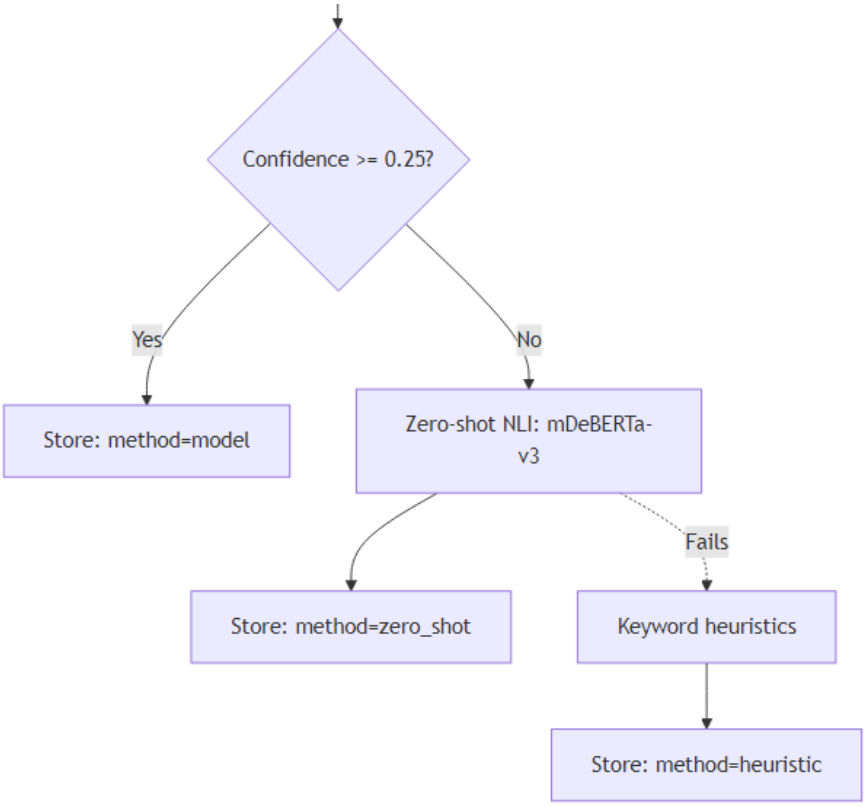



*Figure 4: The Four-Tier NLP Classification Cascade flowchart, demonstrating the path from raw text through normalization, transliteration, section mapping, and multi-model ML inference.*

**Tier 1: Deterministic Legal Section Mapping ( `section_map.py` )**

- **How it works:** Legal codes are ground truth. If the FIR contains explicit BNS or IPC sections (e.g., "302", "379"), the system doesn't need to guess. A massive dictionary maps these codes to categories ( `302 -> murder` , `379 -> theft` ).
- **Conflict Resolution:** If a FIR registers both Murder and Theft, a strict severity priority list resolves the tie: `murder > rape_sexoff > dacoity_robbery > ... > theft` .
- **Result:** If this tier succeeds, the classification is saved with a `confidence` of `1.0` and the method marked as `section_map` .

**Tier 2: The Fine-Tuned NLP Model (IndicBERT / MuRIL)**

- **How it works:** If no sections are found, the text is passed to the fine-tuned 110M parameter neural network.
- **The Confidence Gate:** The model returns softmax probabilities across all 11 categories. A strict environment variable `NLP_CONFIDENCE_THRESHOLD` (default 0.25) acts as a gate. If the model's top prediction is below 25% confident, the platform assumes the AI is guessing blindly and rejects the answer.

**Tier 3: Zero-Shot Natural Language Inference ( `zero_shot.py` )**

- **How it works:** If the fine-tuned model fails the confidence gate, the text falls to a massive, pre-trained Zero-Shot model ( `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli` ).
- **Advantage:** This 270MB model requires *zero* local training data. It understands over 100 languages. It treats classification as a logical hypothesis test (e.g., "This text is about cybercrime. True or False?").
- **Constraint:** It is slow. To prevent it from hanging the CPU for minutes, the input text is hard-truncated to 600 characters before execution.

**Tier 4: Keyword Heuristics**

- **How it works:** The ultimate safety net. If all complex AI fails or crashes, the system scans the text against a hardcoded dictionary of English and Gujarati keywords (e.g., scanning for "મર્ડર" to return "murder").

### 8.3 The AI Mismatch Auditor

A key UI feature: If Tier 1 (Section Map) determines the FIR is a `theft` (based on section 379), but Tier 2 (NLP Model) reads the narrative and strongly believes it is an `assault`, the system writes both to the database. It flags the FIR with `status="review_needed"`. The frontend then displays a massive red "AI Mismatch" warning, forcing a human Senior Officer to review the conflicting data.

---

## 9. Machine Learning Operations (MLOps) and Evaluation Strategy

Machine Learning in the Atlas Platform is not static; it is an iterative loop. As real police data flows through the system, the models must be retrained to improve accuracy.

### 9.1 The Annotation Engine (Label Studio)

As defined in ADR-D04, law enforcement data cannot be uploaded to cloud annotation services like Scale AI.

- **Local Deployment:** An open-source instance of Label Studio runs via Docker alongside the main backend.
- **Workflow:** The script `batch_import_firs.py` regularly pulls a stratified sample of confusing or mismatched FIRs from PostgreSQL. `setup_labelstudio.py` injects them into the local Label Studio UI. Domain experts manually highlight text and select the correct crime category. The verified "Gold Standard" JSON is exported back to the ML pipeline.

### 9.2 Experiment Tracking (MLflow)

Every time a developer runs the `train.py` script to fine-tune the IndicBERT model, MLflow tracks the parameters (learning rate, epochs, batch size) and the resulting metrics (validation loss, F1 score). It stores the `.bin` checkpoint files locally in `atlas_mlflow_data`, creating an auditable history of the AI's evolution.

### 9.3 The Evaluation CI/CD Gate (ADR-D05)

Before a newly trained model is allowed to process real police FIRs, it must pass an automated evaluation script ( `evaluate.py` ).

- **The Metric:** The system uses **Macro-Averaged F1 Score**, not raw accuracy. This is crucial because "Theft" makes up 60% of all crime. Raw accuracy would look amazing even if the AI failed to identify a single "Kidnapping". Macro-F1 forces the AI to prove it can identify rare crimes equally well.
- **The Gate:** If the test suite detects a Macro-F1 score below 0.80, the script throws an exit code 1, automatically failing the deployment pipeline.
- **Bias Auditing:** A secondary script ( `bias_report.py` ) runs geographic Chi-square tests. It ensures the AI isn't accidentally biased against FIRs coming from specific rural districts due to dialect differences.

---

## 10. Infrastructure, Deployment, and Observability

### 10.1 Containerized Deployment

The entire platform is defined as infrastructure-as-code inside `docker-compose.yml`. To deploy a new district hub server, an administrator simply clones the repository, mounts the `.env` secrets file, and runs:

```
docker compose up -d --build
```

This spins up:

- The FastAPI Python Backend
- The Next.js Node Frontend
- PostgreSQL 15, MongoDB 7, and Redis 7
- Prometheus, Grafana, MLflow, and Label Studio

### 10.2 System Observability

- **Prometheus:** Continuously scrapes memory usage, CPU load, and API endpoint latency from the FastAPI backend.
- **Grafana:** Provides visual dashboards. System administrators can monitor the exact millisecond latency of the `ATLASClassifier`, track the queue depth of pending OCR jobs, and spot spikes in HTTP 500 errors immediately.

---

## 11. Roadmap: External API Integrations

While currently air-gapped and self-contained, the `src/integrations` directory holds the structural scaffolding for future Phase 2 interoperability. Once secure API gateways are established by the state government, the platform will interface with:

- **eGujCop:** Two-way sync of FIR status and IO assignments.
- **e-Prison:** Pulling real-time custody status of identified accused individuals.
- **ICJS (Inter-operable Criminal Justice System):** Pushing aggregated, anonymized crime statistics to national dashboards.
- **NCRB (National Crime Records Bureau):** Automated compliance reporting.

---

## 12. Conclusion

The Atlas Platform stands as a testament to building high-security, AI-native software under extreme constraints. By rejecting fragile microservices in favor of a robust modular monolith, enforcing absolute data privacy at the database-query level, and engineering a highly fault-tolerant, four-tier NLP classification cascade, the architecture successfully brings modern automation to the complex, multilingual landscape of Indian law enforcement data.